



Visão Computacional e Deep Learning

Ma. Cilene Renata Real
cilene.real@facens.br

Agenda

- Alinhamento
- Viés Intuitivo
- Vision Transformers (ViT)
- Detecção de objetos (YOLOv8)
- Segmentação

1-) Postei na terça-feira (09/06) a gravação das aulas e um aviso informando vocês sobre elas.

Conseguiram acessar? Algum problema?

2-) A diretoria da pós-graduação nos orientou para dispensarmos todos os alunos hoje às 16h, em virtude do jogo do Brasil.

Considerando essa solicitação, alterei o método de exercícios, mas também, podemos ajustar o horário do almoço.

Viés indutivo são as suposições que um modelo faz para ajudar a aprender com os dados.

Imagina que estamos montando um quebra-cabeça juntos.

Podemos pensar que as peças se encaixam com base na **forma** e na **cor**.

Essa suposição nos ajuda a juntar as peças mais rápido. **É assim que funciona um modelo CNN.**

Ele faz suposições inteligentes, achando que as partes próximas em uma imagem estão relacionadas.

Essas suposições úteis são chamadas de **fortes vieses indutivos**.

Viés Intuitivo

Agora imagina que nos deram uma grande pilha de peças de quebra-cabeça, mas não fazemos ideia de como será a imagem final.

A gente nem sabe se deve olhar para a forma, a cor ou talvez algo completamente diferente.

Então começamos a tentar tudo e, aos poucos, descobrimos o que funciona.

É isso que um modelo Transformer faz. Não faz suposições.

Ele aprende padrões analisando muitos dados. Isso é chamado de **viés indutivo mínimo**.

Isso dá mais liberdade ao modelo, mas **precisa de mais dados e tempo para aprender**.

Vision Transformers (ViT)

Até 2020, o mundo da visão computacional era dominado por **Redes Neurais Convolucionais (CNNs)**.

O problema das CNNs é que elas enxergam localmente.

Cada filtro olha apenas uma pequena região da imagem por vez.

Para que informações distantes se comuniquem, é preciso empilhar muitas camadas.

O ViT propôs algo radical: **jogar fora as CNNs e usar apenas Transformers.**

A mesma arquitetura que revolucionou o processamento de linguagem natural (GPT, BERT) foi aplicada diretamente em imagens.

Vision Transformers (ViT)

Vision Transformers (ViT) são uma arquitetura de redes neurais para processamento de imagens que adapta o mecanismo de **Transformers**, originalmente desenvolvido para processamento de linguagem natural, para tarefas de visão computacional.

Como funcionam?

Em vez de analisar a imagem inteira de uma vez:

- **Divide a imagem em pequenos blocos (patches)**, por exemplo, de 16×16 pixels.
- **Transforma cada patch em um vetor numérico (embedding)**.
- **Adiciona informações de posição**, para que o modelo saiba onde cada patch estava na imagem.
- **Processa esses vetores com camadas Transformer**, utilizando o mecanismo de **self-attention**, que permite ao modelo entender as relações entre diferentes regiões da imagem.
- **Produz uma representação global** da imagem, usada para classificação, detecção de objetos, segmentação, entre outras tarefas.

Patches: Cortando a Imagem em Pedacos:

A primeira pergunta é: **como alimentar uma imagem em um Transformer?**

O Transformer trabalha com **sequências de tokens** (como palavras em uma frase). Uma imagem é uma grade de pixels, e não uma sequência.

A solução do ViT é simples e engenhosa: **dividir a imagem em uma grade de quadrados**, chamados de **patches**.

Cada patch é um pedaço da imagem, como uma peça de quebra-cabeça.

Uma imagem de 224×224 pixels dividida em patches de 16×16 gera uma grade de 14×14 , ou seja, **196 patches**. Cada patch captura uma região local da imagem: um olho, um pedaço do fundo, parte de uma pata.

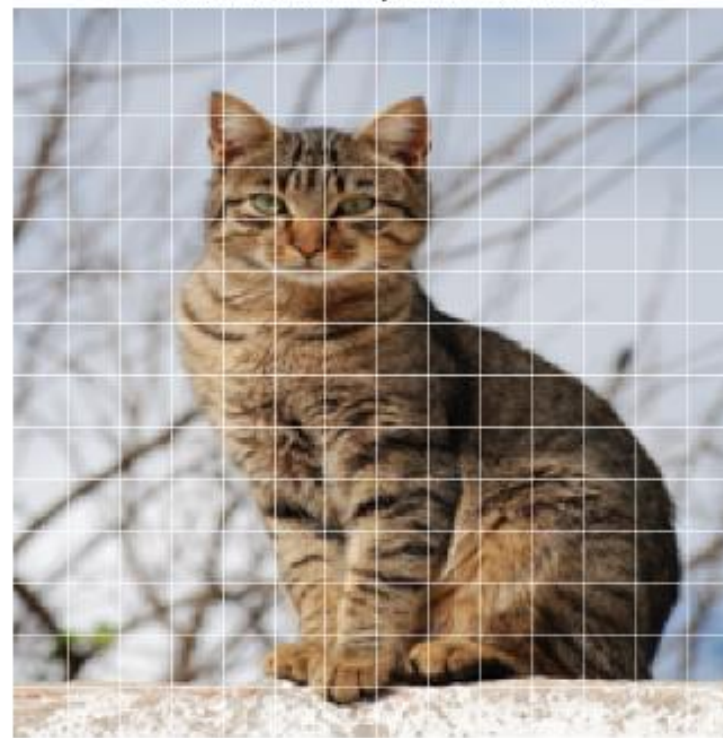
Vision Transformers (ViT)

Se o Transformer de texto processa uma sequência de palavras, o ViT processa uma **sequência de patches**. Cada patch é um “token visual”.

Imagem original



Dividida em 196 patches (16x16)



Por gentileza, abram o Colab e criem um novo notebook.

Copiem o arquivo imagem1.jpg para o seu google drive.

Insira o código com as configurações iniciais:

```
#Configurações Iniciais
import pandas as pd #importa a biblioteca pandas, que é essencial para manipulação e análise de dados em Python.
from google.colab import drive # importa a função drive do módulo google.colab
drive.mount('/content/drive') #conectar seu Google Drive ao ambiente do Colab

import os #importa o módulo os
os.chdir("/content/drive/") # muda o diretório de trabalho atual para /content/drive/
!ls # exibirá todos os arquivos e pastas que estão diretamente dentro do seu Google Drive montado.
```

Na sequência, vamos inserir o código que utilizamos na primeira aula, para exibir a imagem, sua largura e altura.

```
import cv2
from google.colab.patches import cv2_imshow # Importa a função patched para Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/imagem1.jpg')

# Verifica se a imagem foi carregada corretamente
if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    print('Largura em pixels: ', end='')
    print(imagem.shape[1]) #largura da imagem
    print('Altura em pixels: ', end='')
    print(imagem.shape[0]) #altura da imagem
    cv2_imshow(imagem) # Usa cv2_imshow para exibir a imagem no Colab
    # cv2.waitKey(0) # waitKey e destroyAllWindows não são necessários com cv2_imshow
    # cv2.destroyAllWindows()

    cv2.imwrite("saida.jpg", imagem)
```

Vision Transformers (ViT)

No código abaixo estamos exibindo os dados da divisão que faremos:

Tamanho da imagem: 320 x 320
Número de patches: 400
Shape dos patches: (400, 16, 16, 3)

```
import numpy as np
from PIL import Image

# Carregar imagem
imagem = cv2.imread('/content/drive/My Drive/Visao/imagem1.jpg')
img = np.array(imagem)

patch_size = 16

altura, largura, canais = img.shape

patches = []

for y in range(0, altura - patch_size + 1, patch_size):
    for x in range(0, largura - patch_size + 1, patch_size):
        patch = img[y:y+patch_size, x:x+patch_size]
        patches.append(patch)

patches = np.array(patches)

print("Tamanho da imagem:", largura, "x", altura)
print("Número de patches:", len(patches))
print("Shape dos patches:", patches.shape)
```

Vision Transformers (ViT)

O código abaixo estamos dividindo a imagem em pequenos pedaços (patches) e exibindo toda a divisão, assim, ele irá demorar alguns minutos para executar.

Copie todos os códigos na sequência:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# Carregar imagem
imagem = cv2.imread('/content/drive/My Drive/Visao/imagem1.jpg')
img = np.array(imagem)

# Informações já obtidas anteriormente
patch_size = 16
altura, largura, canais = img.shape

n_linhas = altura // patch_size # 20
n_colunas = largura // patch_size # 20

# Extrair patches
patches = []
```

Parte 2:

```
for i in range(n_linhas):
    for j in range(n_colunas):
        patch = img[
            i*patch_size:(i+1)*patch_size,
            j*patch_size:(j+1)*patch_size
        ]
        patches.append(patch)

patches = np.array(patches)

print("Tamanho da imagem:", largura, "x", altura)
print("Número de patches:", len(patches))
print("Shape dos patches:", patches.shape)

# Mostrar os patches
fig, axes = plt.subplots(n_linhas, n_colunas, figsize=(15, 15))

indice = 0
```

Parte 3:

```
indice = 0

for i in range(n_linhas):
    for j in range(n_colunas):
        axes[i, j].imshow(patches[indice])
        axes[i, j].set_title(f"({i},{j})", fontsize=5)
        axes[i, j].axis('off')
        indice += 1

plt.tight_layout()
plt.show()
```

No próximo código, visualizamos apenas os 14 patches iniciais:

```
fig, axes = plt.subplots(2, 7, figsize=(14, 4))

for k, ax in enumerate(axes.flat):
    i = k // n_colunas
    j = k % n_colunas

    ax.imshow(patches[k])
    ax.set_title(f"({i},{j})", fontsize=8)
    ax.axis('off')

plt.tight_layout()
plt.show()
```


Patch Embedding: De Pixels para Vetores

Cada patch é uma matriz de pixels (16x16x3 para imagens).

Mas o Transformer trabalha com vetores de dimensão fixa.

Precisamos converter cada patch em um **vetor numérico** (isso é o *patch embedding*).

Na prática, o ViT usa uma **convolução com kernel e stride iguais ao tamanho do patch**.

Essa operação faz duas coisas de uma vez: divide a imagem em patches e projeta cada patch para um vetor de dimensão D.

Vamos executar o código:

```
import torch.nn as nn

# Conv2d com kernel=16 e stride=16: divide e projeta em uma operação
embed_dim = 768 # dimensão do ViT-Base
patch_embed = nn.Conv2d(3, embed_dim, kernel_size=16, stride=16)
```

Podemos reescrever o código de uma maneira diferente:

```
import torch
import torch.nn as nn

# Simulando uma imagem RGB 224x224
x = torch.randn(1, 3, 224, 224)

# Patch Embedding do ViT-Base
embed_dim = 768
patch_embed = nn.Conv2d(
    in_channels=3,
    out_channels=embed_dim,
    kernel_size=16,
    stride=16
)

# Saída da convolução
y = patch_embed(x)

print("Após convolução:", y.shape)
# (1, 768, 14, 14)

# Reorganizar para sequência de patches
y = y.flatten(2).transpose(1, 2)

print("Após flatten + transpose:", y.shape)
# (1, 196, 768)
```

Agora vamos exibir de fato o vetor gerado para o 1º patch.

```
print(y[0, 0])
```

Após, verificar, abram um novo código e agora exibam para 5 patches:

```
for i in range(5):  
    print(f"\nPatch {i}:")  
    print(y[0, i])
```

O resultado é uma sequência de 196 vetores, cada um com 768 dimensões.

Cada patch de 16x16x3 pixels (768 valores) virou um vetor de 768 dimensões – uma representação numérica compacta daquela região da imagem.

CLS Token e Positional Embedding

Antes de entrar no Transformer:

O **CLS token** (de *classification*) é um vetor especial inserido no início da sequência. Ele funciona como um “observador” que não pertence a nenhum patch específico. Ao passar por todas as camadas de atenção, o CLS token vai acumular informação de todos os patches. No final, é ele que usamos para classificar a imagem.

O **Positional Embedding** resolve um problema fundamental: o Transformer não tem noção de ordem.

Sem ele, o modelo não saberia que o patch do canto superior esquerdo é diferente do canto inferior direito. Somamos um vetor de posição a cada token, dando ao modelo a informação de “onde” cada patch estava na imagem original.

A sequência resultante tem **197 tokens** (1 CLS + 196 patches), cada um com 768 dimensões. Essa é a entrada do Transformer.

Self-Attention: A Peça Central

O mecanismo de ***self-attention*** é o que torna o Transformer tão poderoso.

Ele permite que cada token “olhe” para todos os outros tokens e decida a quem prestar atenção.

A analogia mais útil é a de uma **sala de aula**. Imagine que cada patch é um aluno. Cada aluno gera três coisas:

Query (Q): “O que eu estou procurando?”

Key (K): “O que eu tenho para oferecer?”

Value (V): “Qual informação eu carrego?”

Vision Transformers (ViT)

A atenção é calculada comparando a Query de cada token com as Keys de todos os outros.

Quanto mais compatíveis Q e K forem, mais atenção um token dá ao outro.

O resultado é uma **média ponderada dos Values**.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

A divisão por $\sqrt{d_k}$

evita que os valores fiquem grandes demais antes do *softmax*.

É um detalhe simples, mas crucial para a estabilidade do treinamento.

Vision Transformers (ViT)

Na prática, o ViT não usa uma única *attention*, usa **Multi-Head Attention**.

A ideia é dividir o embedding em múltiplas “cabeças” que operam em paralelo, cada uma aprendendo a prestar atenção em **aspectos diferentes**: uma cabeça pode focar em cores, outra em formas, outra em texturas.

As saídas são concatenadas no final.

O Bloco Transformer

Cada bloco do Transformer combina duas operações, ambas com conexões residuais e Layer Normalization:

- **Multi-Head Self-Attention** — os tokens se comunicam entre si.
- **Feed-Forward Network (FFN)** — cada token é processado individualmente por duas camadas lineares.

Vision Transformers (ViT)

O ViT-Base empilha **12 desses blocos** em sequência.

A cada camada, os tokens ficam mais ricos em informação contextual.

O CLS token, que começou sem informação, vai progressivamente “ouvindo” todos os patches e construindo uma representação global da imagem.

As conexões residuais (somar a entrada com a saída) facilitam o fluxo de gradientes durante o treinamento e permitem que o modelo empilhe muitas camadas sem degradar o sinal.

Na Prática: Classificando uma Foto de Gato

A verdade é que, no dia a dia, ninguém implementa o ViT do zero.

Usamos modelos pré-treinados e fazemos inferência ou *fine-tuning*.

O torchvision já inclui o ViT-B/16 pré-treinado no ImageNet com 1000 classes.

Vamos carregar o modelo e classificar a imagem do gato.

Inserir no próximo código:

```
import torch
import torch.nn.functional as F
from PIL import Image
from torchvision.models import vit_b_16, ViT_B_16_Weights

import torch
import torch.nn.functional as F
from PIL import Image
from torchvision.models import vit_b_16, ViT_B_16_Weights

# Carregar a imagem
img = Image.open("/content/drive/My Drive/Visao/imagem1.jpg").convert("RGB")

# Carregar modelo pré-treinado no ImageNet
weights = ViT_B_16_Weights.IMAGENET1K_V1
model = vit_b_16(weights=weights)
model.eval()
```

Parte 2:

```
# Pré-processar a imagem
preprocess = weights.transforms()
img_input = preprocess(img).unsqueeze(0)

# Inferência
with torch.no_grad():
    output = model(img_input)
    probabilidades = F.softmax(output[0], dim=0)

# Top-5 predições
top5_prob, top5_idx = torch.topk(probabilidades, 5)
categorias = weights.meta["categories"]

for i in range(5):
    print(f"{categorias[top5_idx[i]]:30s} {top5_prob[i].item():.1%}")
```


Cada região da imagem é reconstruída como uma combinação ponderada das informações vindas de todas as outras regiões.

Essa capacidade de relacionar diretamente partes distantes da imagem é uma das principais diferenças entre os **Vision Transformers** e as **CNNs tradicionais**, que aprendem relações globais de forma mais gradual através das convoluções.

Compreensão global da imagem

Uma CNN tradicional aprende primeiro características locais:

- bordas;
- texturas;
- formas simples;

E gradualmente combina essas informações para formar objetos maiores.

Já um Vision Transformer permite que uma região da imagem interaja diretamente com qualquer outra região desde a primeira camada.

Por que utilizar Vision Transformers?

Mesmo que essas partes estejam distantes entre si.

Por exemplo, na foto de do cachorro: Cabeça $\leftrightarrow$ Corpo $\leftrightarrow$ Cauda



Mesmo que as partes estejam distantes entre si, haverá a relação entre elas.

Vision Transformers é útil em imagens complexas.

1. Classificação de imagens

Exemplos:

- gato ou cachorro;
- tipos de plantas;
- defeitos em peças industriais.

Entrada:



Saída: Cachorro

2. Detecção de objetos

Encontrar:

- carros;
- pessoas;
- placas;
- animais.

Aplicações:

- veículos autônomos;
- monitoramento;
- robótica.

3. Segmentação de imagens

Separar pixel por pixel:

- tumor;
- estrada;
- céu;
- vegetação.

Muito usada em:

- medicina;
- agricultura;
- imagens de satélite.

Aplicações:

- ressonância magnética;
- tomografia;
- mamografia;
- radiografia.

Objetivos:

- detectar tumores;
- classificar doenças;
- segmentar órgãos.

Veículos autônomos

O sistema precisa reconhecer:

- pedestres;
- carros;
- faixas;
- semáforos.

Modelos baseados em Transformers são amplamente estudados para percepção visual em carros autônomos.

Sensoriamento remoto

Imagens de satélite para:

- monitoramento ambiental;
- agricultura;
- detecção de incêndios;
- mapeamento urbano.

Reconhecimento facial

Identificação de:

- pessoas;
- expressões;
- emoções.

Modelos multimodais: Texto + imagem.

Por exemplo:

Pergunta:

"O que há nessa foto?"

Imagem:



Saída:

"Há um cachorro correndo em um parque."

Essa é uma das áreas em que o Vision Transformers está atuando com grande intensidade.

Nem sempre eles são a melhor opção.

CNNs ainda podem ser melhores quando:

- há pouco dado disponível;
- o hardware é limitado;
- é necessária baixa latência;
- o problema é relativamente simples.

Vision Transformers são particularmente valiosos quando se deseja construir modelos de visão computacional de alto desempenho, especialmente em ambientes com grandes volumes de dados e quando a compreensão global da imagem é importante.

Comparativo

Cenário	Melhor escolha
Poucos dados	CNN
Dispositivo móvel	CNN
Tempo real com pouco hardware	CNN
Grandes bases de dados	ViT
Máxima precisão	ViT
Modelos multimodais	ViT
Segmentação e detecção avançadas	ViT

Execute o último código para:

- imagem2.jpeg
- imagem3.jpeg

Atenção que a extensão dos arquivos mudou.

Avalie se o resultado foi satisfatório ou não.

Imagem 2:

seashore	21.8%
dock	8.2%
liner	4.9%
container ship	4.8%
pier	4.4%

Não satisfatório!

**É uma imagem da
cidade de Sorocaba.**

- A classe mais provável é "**seashore**" (**praia/costa**), com apenas **21,8%** de confiança.
- As probabilidades das demais classes são relativamente próximas e baixas.
- Isso sugere que o modelo **não está muito confiante** em uma única categoria.
- Há um padrão comum entre as cinco classes: todas estão relacionadas a **ambientes marítimos** (praia, cais, doca, navios).

Imagem 3:

planetarium	53.6%
park bench	17.3%
dome	3.6%
radio telescope	1.6%
shopping cart	1.4%

Não satisfatório!

É uma imagem de
algumas árvores.

- **53,6% para "planetarium"** indica uma confiança moderadamente alta. O modelo vê características que lembram um planetário, provavelmente uma **estrutura arredondada ou uma grande cúpula**.
- **17,3% para "park bench"** sugere que algum elemento da imagem possui forma ou textura semelhante a um banco de parque.
- **"dome" (3,6%)** reforça a ideia de que há uma forma esférica ou arredondada na imagem.
- **"radio telescope" (1,6%)** pode indicar que certos padrões ou estruturas lembram equipamentos astronômicos.
- **"shopping cart" (1,4%)** tem probabilidade muito baixa e pode ser considerado um palpite secundário do modelo.

Para resolver esse problema, mas utilizar um outro modelo.

Para cenas, cidades, árvores e objetos pouco comuns, o CLIP.

CLIP é um modelo de inteligência artificial desenvolvido pela OpenAI em 2021. O nome vem de Contrastive Language–Image Pretraining.

A ideia central do CLIP é aprender a relacionar imagens e texto.

Enquanto modelos tradicionais como ResNet ou ViT treinados no ImageNet aprendem a classificar imagens em um conjunto fixo de categorias (1000 classes), o CLIP aprende a associar imagens a descrições em linguagem natural.

Ele foi treinado com cerca de **400 milhões de pares imagem-texto** encontrados na Internet. Por isso, adquiriu um conhecimento visual muito mais amplo do que os modelos tradicionais treinados apenas no ImageNet.

Implemente o código abaixo para a imagem2 e imagem3 e vamos analisar os resultados.

```
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch

image = Image.open("/content/drive/My Drive/Visao/imagem3.jpeg")

model = CLIPModel.from_pretrained("openai/clip-vit-large-patch14")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-large-patch14")

labels = [
    "city skyline",
    "urban landscape",
    "oak tree",
    "palm tree",
    "forest",
    "park",
    "beach",
    "mountain"
]
```

Parte 2:

```
inputs = processor(  
    text=labels,  
    images=image,  
    return_tensors="pt",  
    padding=True  
)  
  
outputs = model(**inputs)  
probs = outputs.logits_per_image.softmax(dim=1)  
  
for label, p in zip(labels, probs[0]):  
    print(f"{label:20s} {100*p.item():.2f}%")
```

**Com esse modelo os resultados
foram favoráveis?**

Imagem 2:

Categoria original	Tradução	Probabilidade
city skyline	Horizonte da cidade / Silhueta urbana	78,69%
urban landscape	Paisagem urbana	21,03%
oak tree	Carvalho	0,01%
palm tree	Palmeira	0,11%
forest	Floresta	0,02%
park	Parque	0,04%
beach	Praia	0,00%
mountain	Montanha	0,10%

O CLIP está indicando que, entre as descrições que você forneceu, a imagem corresponde principalmente a uma paisagem urbana, assim, foi satisfatório.

Imagem 3:

Categoria original	Tradução	Probabilidade
city skyline	Horizonte da cidade / Silhueta urbana	0,08%
urban landscape	Paisagem urbana	3,04%
oak tree	Carvalho	1,46%
palm tree	Palmeira	0,03%
forest	Floresta	3,00%
park	Parque	92,28%
beach	Praia	0,01%
mountain	Montanha	0,09%

A imagem se parece predominantemente com um parque, contendo possivelmente árvores e vegetação, enquanto características de cidade, praia ou montanha são muito pouco evidentes. Também foi satisfatório.

Exercício

Agora peguem a imagem que vocês utilizaram na primeira aula (sua imagem ou da família), e repita o processo para os dois modelos e avaliem o resultado.

Realizem a mudança das categorias no CLIP:

```
labels = [  
    "person",  
    "man",  
    "woman",  
    "child",  
    "group of people",  
    "portrait",  
    "face",  
    "selfie"  
]
```

Considerando a minha foto:



O Vit retornou:

Categoria original	Tradução	Probabilidade
lipstick	Batom	19,6%
hair spray	Spray de cabelo	6,3%
face powder	Pó facial / maquiagem em pó	3,9%
perfume	Perfume	3,5%
hand blower	Secador de mãos	3,3%

Exercício

O CLIP retornou:

Categoria original	Tradução	Probabilidade
person	Pessoa	5,84%
man	Homem	0,42%
woman	Mulher	36,60%
child	Criança	1,03%
group of people	Grupo de pessoas	1,14%
portrait	Retrato	14,75%
face	Rosto	20,19%
selfie	Selfie	20,05%

Conclusão

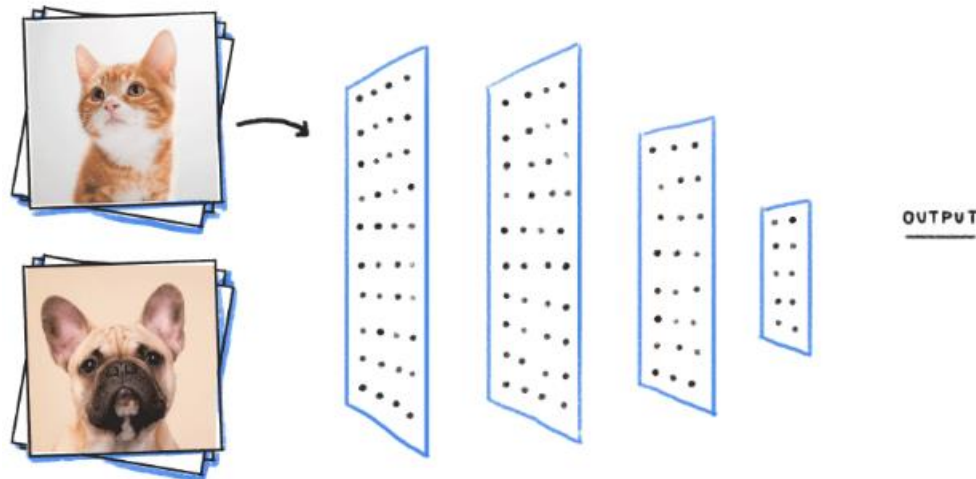
Com esses exemplos e exercícios, entendemos que existem diversos modelos e devemos utilizar aquele que for adequado ao contexto.

Tarefa	Modelo recomendado
Detectar pessoas	YOLOv8
Detectar rostos	RetinaFace
Reconhecimento facial	ArcFace / FaceNet
Estimar idade e gênero	DeepFace
Analisar postura	MediaPipe Pose
Segmentar a pessoa	SAM 2 / YOLOv8-seg
Descrever a pessoa em linguagem natural	Florence-2
Classificação por texto	CLIP ou SigLIP
Detectar "homem com chapéu", "mulher sorrindo" etc.	Grounding DINO
Solução multimodal mais completa atualmente	Florence-2

Detecção de objetos

Diferente da classificação, que busca apenas prever a classe presente na imagem, a detecção de objetos além de prever qual é a classe precisa identificar também a localização do objeto nessa imagem.

Por exemplo, quando construímos um classificador de cachorro e gatos, o nosso classificador vai retornar como resultado da predição a classe (cachorro ou gato) e a confiança (grau de certeza sobre aquela predição).



Mas e se a imagem tiver tanto o gato quanto o cachorro?

Poderíamos talvez treinar classificadores multi-classe que vão ser capazes de identificar as duas classes (tanto cachorro como gato).

No entanto, nós ainda não temos como saber a localização do cachorro ou gato na imagem.

Esse problema de identificar a localização de um objeto (dada a classe) em uma imagem é chamado de localization (**localização**).

Detecção de objetos

Ser capaz de prever a localização do objeto na imagem junto com a classe do objeto (para múltiplas classes) é o que chamamos de **Detecção de Objetos**.



O retângulo que contém a localização do objeto é chamado de **caixa delimitadora** (bounding box).

YOLO é um método de detecção de objetos de passada única (single pass) que utiliza uma rede neural convolucional como extrator de características (features).

Diferente de algoritmos anteriores de detecção de objetos, como R-CNN ou Faster R-CNN, ele apenas precisa olhar pela imagem uma única vez para enviar para a rede neural.

Por isso ele recebe esse nome (You Only Look Once – “Você só olha uma vez”).

E devido a essa característica, o YOLO foi capaz de conseguir uma velocidade na detecção muito maior do que as técnicas concorrentes, sem perder em acurácia.

Funcionamento

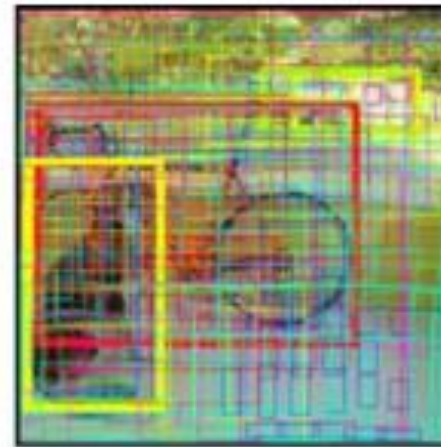
O YOLO trata a detecção de objetos como um simples problema de regressão.



1) grid de 13x13



2) quanto maior a confiança, mais grossa é a linha representada



3) é feito a previsão da classe para cada caixa delimitadora



4) apenas são mantidas as caixas acima do limiar definido

1. Primeiro, o algoritmo divide a imagem em um grid de $S \times S$ células.

Como exemplo usaremos 13×13 , porém esse tamanho pode mudar.

Para as versões mais recentes por exemplo tem-se preferido utilizar um grid de 19×19 .

2. Cada uma dessas células é responsável por fazer a predição de 5 caixas delimitadoras (B), para caso haja mais de um objeto naquela célula.

Também é retornado a pontuação de confiança que nos diz o quanto de certeza ele tem que aquela caixa delimitadora contenha um objeto.

3. Para cada caixa, a célula também faz a previsão de uma classe.

Isso funciona como se fosse um classificador: é fornecido um valor de probabilidade para cada uma das classes possíveis.

O valor de confiança para a caixa delimitadora e a predição da classe são combinados em uma pontuação final, que vai nos dizer a probabilidade dessa caixa conter um objeto específico.

Nesse caso o grid é 13×13 , o que no final resulta em 169 células.

Para cada uma dessas células são detectadas 5 caixas delimitadoras, o que resulta em 845 no total.

4. Acontece que a maioria dessas caixas terá um valor de confiança extremamente baixo, então por isso geralmente se considera apenas as caixas cuja pontuação final seja 30% ou mais.

Esse valor de 30% é o limiar, chamado de **threshold**, e ele pode ser alterado dependendo do quão preciso você quer que o detector seja.

Darknet

Para o seu funcionamento o YOLO utiliza uma rede neural profunda, cuja arquitetura é chamada de Darknet, que é o mesmo nome do framework utilizado para implementar o detector.

Esse framework foi desenvolvido pelo próprio criador do YOLO, Joseph Redmon.

Ele é open source e escrito na linguagem C, também possui suporte para GPU.

O YOLO comparado a outras técnicas de detecção de objetos

Ao longo dos anos foram desenvolvidas diversas abordagens e metodologias para resolver esse mesmo problema da detecção de objetos.

A técnica Haar Cascades, foi considerada a primeira abordagem a conseguir resultados realmente satisfatórios e que pudessem ser implementados em aplicações reais, sendo até hoje ainda bastante utilizada.

Poucos anos depois, em 2005, é apresentado o HOG – Histogram of Oriented Gradients, o que também foi considerado um marco importante para essa área.

Mais tarde, com a ascensão do Deep Learning, foram desenvolvidos métodos baseados em Redes Neurais Convolucionais (CNN).

Dentre os principais, podemos citar:

- R-CNN – Region-based Convolutional Neural Networks
- SPP-net – Spatial Pyramid Pooling
- Fast R-CNN
- Faster R-CNN

E onde o YOLO se encaixa?

O YOLO veio logo depois, junto com o Faster R-CNN.

A vantagem do YOLO frente aos outros métodos é que esse faz as predições da classe com uma única passada na rede.

Antes dele, esses outros principais sistemas de detecção de objetos faziam a detecção através da divisão da imagem em várias partes e depois em cada pedaço da imagem se executava um classificador, em cada uma dessas regiões (como é o caso do Haar Cascade por exemplo).

As pontuações altas retornadas por esse classificador eram consideradas detecções na imagem (ou seja, os próprios objetos).

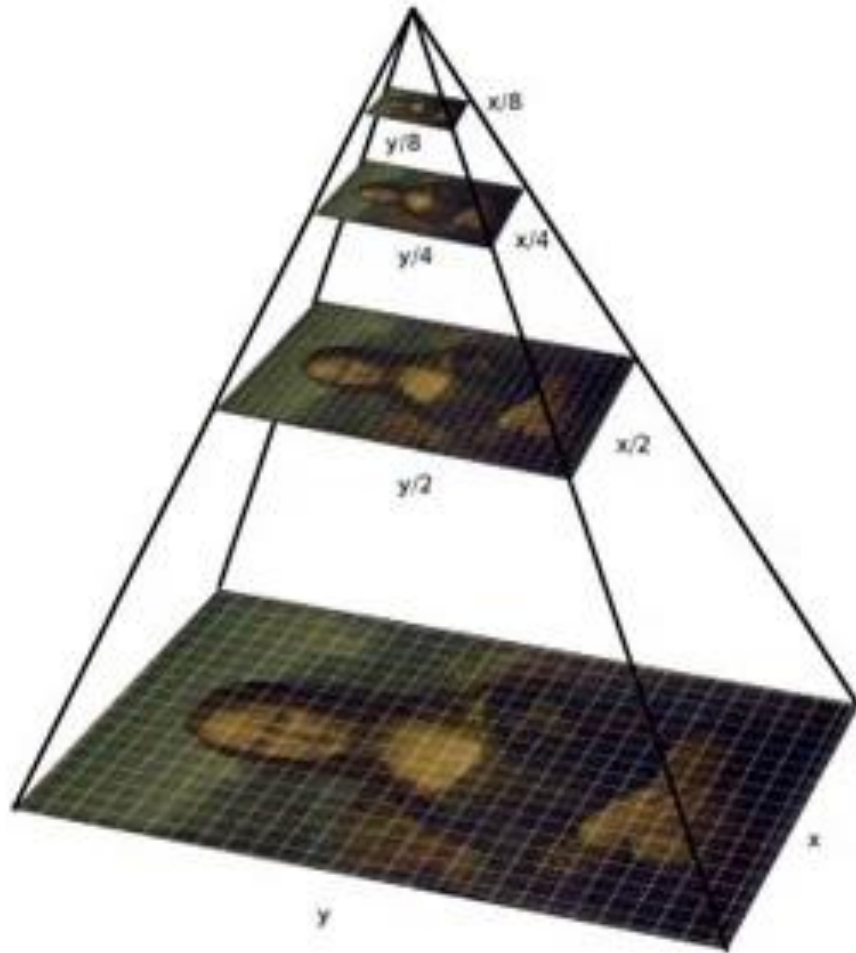
E onde o YOLO se encaixa?

Podemos usar um classificador como VGGNet ou Inception e transformá-lo em um detector de objetos se usarmos uma janela que se desloca pela imagem (sliding window).

Em cada etapa é rodado um classificador para obter uma predição a respeito de que tipo de objeto é dentro da janela.

Usando a janela deslizante pode-se retornar centenas ou milhares de predições para aquela imagem, mas mantemos apenas aquelas que o classificador possui uma certeza maior.

E onde o YOLO se encaixa?



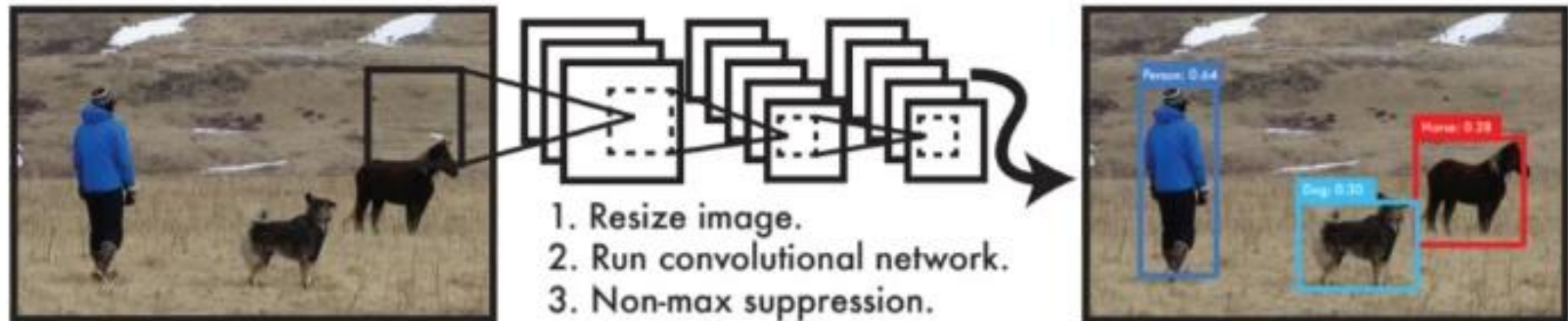
Isso significa que é necessário rodar o mesmo classificador dezenas ou milhares de vezes sobre a mesma imagem.

Mas diferente de quando estamos usando uma abordagem clássica como o Haar Cascade, ao lidarmos com uma técnica que utiliza redes neurais artificiais (ou seja, baseada em Deep Learning) é muito demorado e pesado executar a mesma operação sobre a imagem milhares de vezes, ainda mais se for rodar isso em condições normais usando um computador convencional.

E onde o YOLO se encaixa?

Comparando com esses outros métodos, o YOLO possui uma abordagem completamente diferente.

Ele não é um classificador tradicional que foi reajustado para ser um detector de objetos, mas sim foi feito para ser capaz de rodar por toda a imagem apenas de uma vez, fazendo isso de uma forma inteligente.



Instalar o YOLO:

```
!pip install ultralytics
```

Importar o modelo:

```
from ultralytics import YOLO
```

Próximo código:

```
model = YOLO('/content/yolov8n.pt')
```

Informar o caminho da imagem:

```
imagem = '/content/drive/MyDrive/Visao/pessoa.jpg'
```

Fazer a detecção:

```
results = model(imagem)
```

Mostrar a imagem com as detecções:

```
from matplotlib import pyplot as plt

# Obtém a imagem com as caixas desenhadas
img_resultado = results[0].plot()

plt.figure(figsize=(10,10))
plt.imshow(img_resultado)
plt.axis('off')
plt.show()
```


Vamos repetir os 3 últimos códigos para as imagens:

- cachorro.jpg
- cavalo.jpg
- geral.jpg
- grito.jpg

Por que será que na imagem geral ele só
detectou a pessoa?

E grito ele detectou a imagem toda?

O YOLO possui foi treinado para 80 classes, e aquilo que ele não conseguiu identificar, não faz das classes existentes no seu treinamento.

Exercício

- 1-) Execute os códigos para o arquivo transito.jpg.
- 2-) Obtenha uma imagem menos futurista e repita os 3 últimos códigos.
- 3-) Por favor, procurem por mais uma imagem de trânsito e apliquem a modificação do modelo proposto pelo Bruno:

```
# Modelo extra-large (reutiliza se já foi baixado na célula anterior)
model_x = YOLO('/content/yolov8x.pt')
```

```
results = model_x(
    '/content/drive/MyDrive/Visao/transito.jpg',
    conf=0.15,
    imgsz=1280
)
```

4-) Encontre algumas imagens:

- Medicamento
- Placa
- Flor
- Uma imagem com vários elementos distantes uns dos outros.
- Uma imagem com vários elementos próximos.
- Uma imagem escolhida por vocês.

Execute o código do ViT e do CLIP para essas imagens, realize as comparações.

Será que conseguimos encontrar algum outro modelo (slide 54), para fazer a comparação?

5-) Considerando as imagens obtidas no exercício anterior, execute o código do YOLO. Na sequência, peça para uma IA Generativa te ajudar a criar um código para executar um modelo do slide 65 e faça a comparação de execução.